



AI with Rav

Learn AI the way you actually think.



DAY 5 of 30

OPEN SOURCE CONTRIBUTION

Testing the Thing That Quietly Returns False

WHAT YOU'LL LEARN TODAY

- > Why a function's guard clause is its most important behaviour
- > Testing a function that writes files
- > How to test "it did nothing"
- > Two days from open to merge



Testing the Thing That Quietly Returns False

In One Line: A function that silently does nothing when its input is wrong is the easiest thing in a codebase to break by accident. That is exactly why it deserves a test.

The repo

sentence-transformers — the standard library for turning text into embeddings. If you have built semantic search or a RAG pipeline in Python, you have used it.

Same discovery method as Day 4: a utility module with a function that had no tests. This time it was **append_to_last_row** in **util/misc.py**.

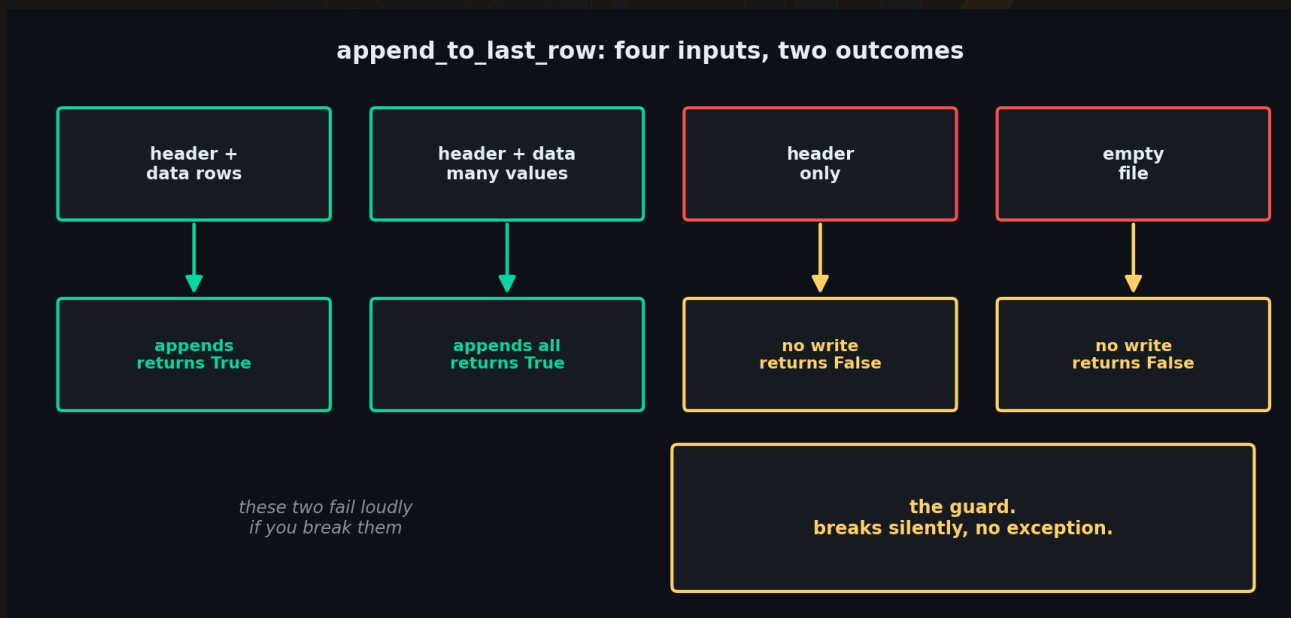
What the function does

It tacks extra columns onto the **last row** of a results CSV. You have a file of evaluation results, you compute one more number, you want it on the end of the most recent row.

Simple — except for one detail that makes it interesting.

The guard

It only writes when the file has a header **and** at least one data row. Otherwise it does nothing and returns **False**.



Think about why that guard exists. If the file is empty, there is no last row to append to. If the file has only a header, the "last row" is the header — appending your values there would corrupt the column names.



So the function refuses. Quietly, with a **False** return.

This is the most dangerous shape of code to leave untested. If a refactor removes that guard, nothing crashes. No exception, no stack trace. You get a results file with values appended to the header row, and you find out days later when a chart looks wrong.

Testing that nothing happened

Most tests assert that something *did* happen. Here the important assertions are that it did **not**:

```
def test_append_to_last_row_header_only(tmp_path):
    path = tmp_path / "results.csv"
    path.write_text("a,b\n")
    before = path.read_text()

    assert append_to_last_row(path, ["x"]) is False
    assert path.read_text() == before  # file untouched
```

Two assertions, and both matter:

- It returned **False** — the caller is told it did not happen
- The **file is byte-for-byte unchanged** — it did not half-write something

The second one is the real test. A broken version could still return **False** while having already written to the file.

The four cases

Input	Expected
Header + data rows	Appends to last row, returns True
Header + data, multiple values	All values appended
Header only	Returns False, file untouched
Empty file	Returns False, file untouched

Two happy paths, two guard paths. That is the whole surface of the function.

Use `tmp_path`. Pytest gives every test its own throwaway directory, so tests that write files stay isolated and leave nothing behind. Never write into the repo from a test.

Merged in two days



Opened 11 July, merged 13 July, by **tomaarsen** — the sentence-transformers maintainer.

Compare that with Day 4's three weeks. Same kind of PR, very different pace. That difference is not about quality; it is about who is at their desk that week. You cannot control it, so do not read anything into it.

Reading the function before writing a line

Before writing any of those four tests, I read the function and wrote down what it does in plain sentences. It is worth showing that step, because it is where the test cases actually come from.

- It opens a CSV and reads the rows
- If there are **fewer than two rows**, it stops and returns False
- Otherwise it appends the values to the **final row** and rewrites the file
- It returns **True** when it wrote

Four sentences, four tests. The mapping is almost mechanical once the behaviour is written out — the hard part is resisting the urge to open the editor first.

If you cannot describe a function in four plain sentences, you do not understand it well enough to test it yet. Keep reading. That is not wasted time; it is the work.

What I deliberately did not test

Equally important, and easy to get wrong. I did not write tests for:

Not tested	Why not
File does not exist	That is the OS raising, not this function's behaviour
A locked or unreadable file	Environment-specific and flaky in CI
Enormous files	Slow, and it tests nothing this function decides
Exact CSV quoting rules	That is the csv module's job, not this function's

A test that fails for reasons unrelated to the code under test is worse than no test. It trains everyone to ignore red CI.

The instinct to add "just one more edge case" is how a five-test PR becomes a twenty-test PR that a maintainer will not review. Test what this function decides. Let the standard library test itself.



Why the guard-clause instinct transfers

Once you start looking for them, you find these everywhere:

- Functions that return **early** when input is empty or malformed
- Functions returning a **bool** to signal "I chose not to act"
- **if not x: return** at the top of a function, with **no test** exercising it

Every one of those is a contribution waiting to happen — and more importantly, it is the same instinct that finds real bugs. Day 1's joblib bug was a guard clause that checked the wrong thing.

Look at what a function refuses to do, not just what it does. That is where the untested behaviour is, and it is where the wrong assumptions live.

On tooling

The PR noted **ruff check** and **ruff format** were clean. Every repo has its own checks — **pre-commit**, **ruff**, **black**, **flake8**. Find them in **CONTRIBUTING.md** or the pre-commit config, run them before you push, and say so.

It costs you thirty seconds and removes the most common reason a PR gets a "please fix the linter" comment instead of a merge.

What is next

Two videos left, and they cover the kind of PR I opened thirty-five of before any of the above. Most of them never merged. The next one explains what separates the ones that did.

